Custom Search, Display data in popup, Local storage

This comprehensive guide breaks down how to implement efficient custom searches, leverage dynamic popups, and master FlutterFlow's internal state management systems.

1. Implementing Custom Search (Simple Search)

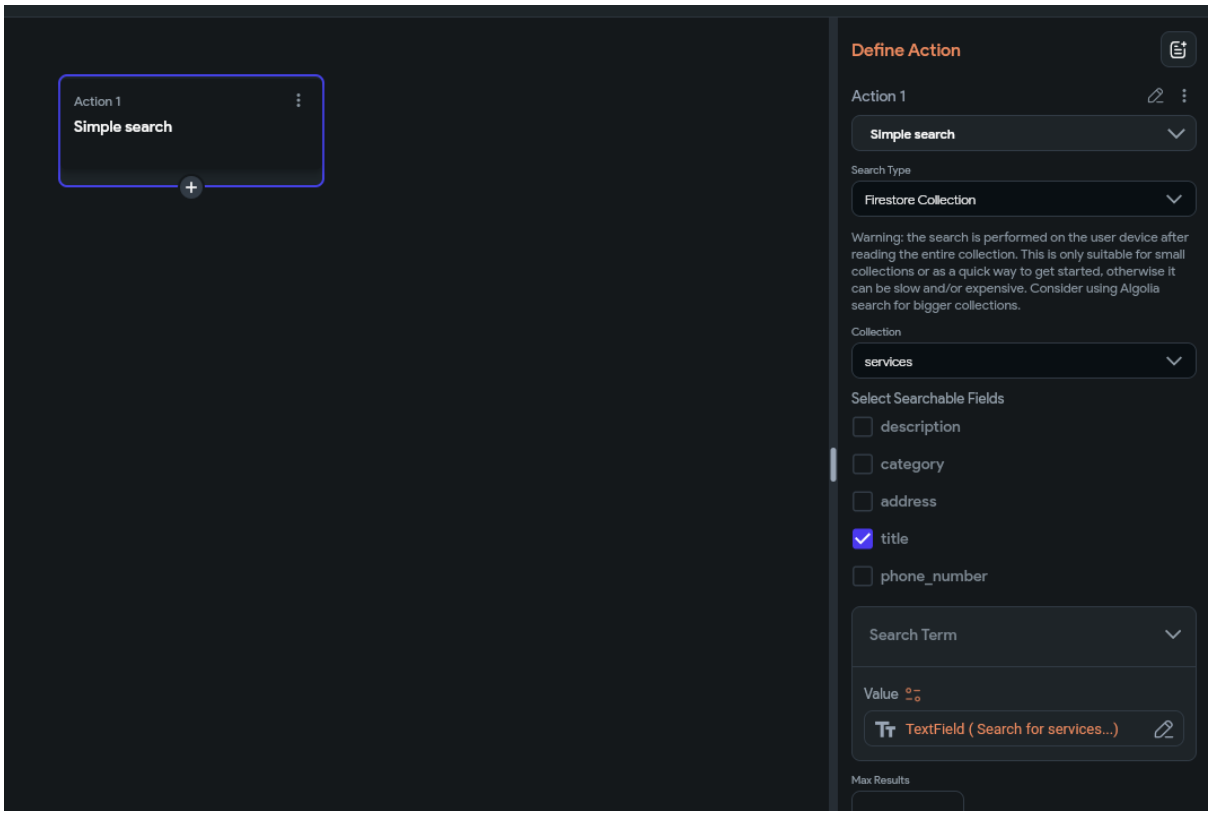
FlutterFlow offers a built-in **Simple Search** action. Unlike server-side search engines like Algolia (which query external databases), Simple Search executes entirely **on the user's device**. It filters data that has already been loaded into the application's memory.

⚠️ **Performance Rule of Thumb:** Only use Simple Search for smaller datasets (e.g., a few hundred records). For massive databases, use Algolia or native Supabase/Firestore text indexing to avoid lagging user interfaces and high memory consumption.

The Three Types of Simple Search

- **Firestore Collection / Supabase Table:** Filters a raw database table directly.
- **Documents / Rows:** Filters a list of records already retrieved by a parent widget query.
- **Strings:** Filters a basic list of text strings stored in a local variable.

Step-by-Step Implementation



1. **Place your UI Elements:** Add a **TextField** (for the user input) and a **Button** or **IconButton** (to trigger the search) at the top of your page.
2. **Open the Action Flow Editor:** Select your Search Button, navigate to the **Actions Panel** on the right, and click **Open**.

3. **Add the Simple Search Action:** Click **+ Add Action**, search for **Simple Search**, and select it.
4. **Configure the Search Type:** * Choose your source (e.g., **Documents** if you are filtering an existing API or database list asset).
    - Set the **Source** variable to match your page's existing data query list.
    - Under **Searchable Fields**, check the specific database fields you want to scan (e.g., `title`, `description`).
5. **Link the Search Term:** Under **Search Term**, change the source to **Widget State** and select your **TextField**.
6. **Display the Results:** FlutterFlow automatically creates a local output variable from this search. Map your downstream **ListView** or **GridView** to this search output variable rather than the raw database query.

2. Displaying Data in Popups (Bottom Sheets & Dialogs)

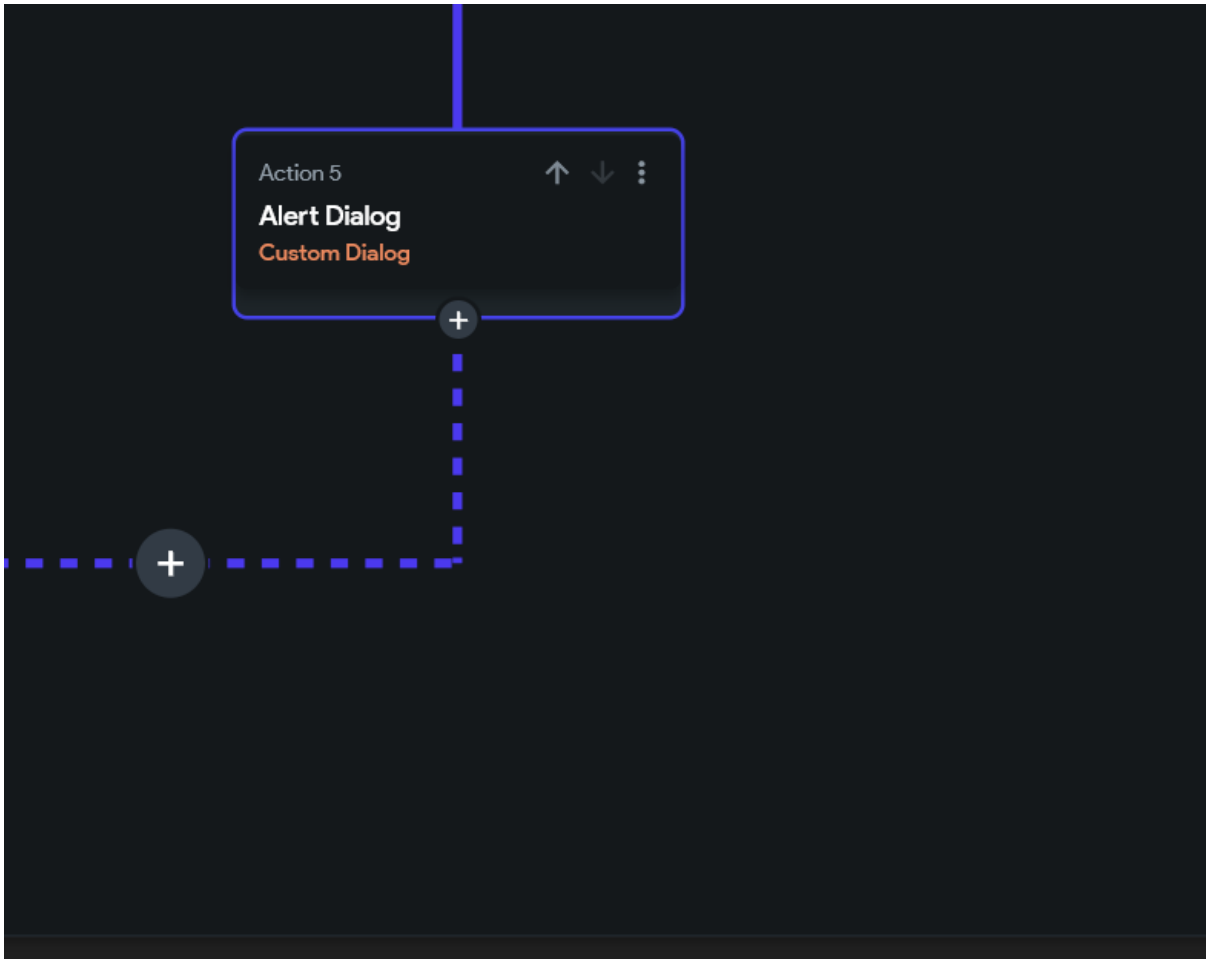
Popups keep users grounded on their current screen while displaying or collecting extra details. In FlutterFlow, popups are built using **Components**.

Step 1: Create the Component

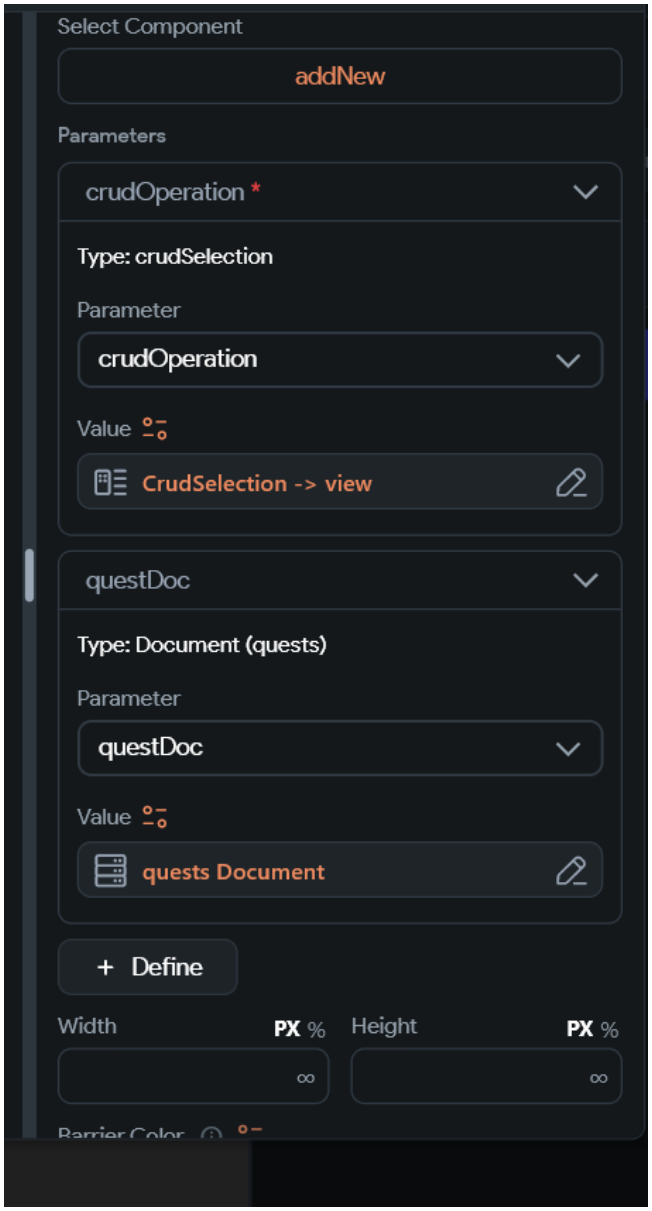
1. Navigate to the **Component Architecture** tab on the left menu (below pages) and click **Create New Component**.
2. Design your popup UI (e.g., a card containing a product image, title, and an "Close" button).
3. **Crucial step for displaying dynamic data:** Go to the Component Settings tab (right menu) and define a **Component Parameter** (e.g., `selectedItem`, Type: JSON or Document). This acts as a gateway to pass data from the main page into the popup.
4. Map the widgets inside your component to this Parameter.

Step 2: Launch the Popup from the Main Page

1. On your main page's ListView, select the item wrapper (like a Container or Card) and open the **Action Flow Editor**.
2. Add a new action: **Bottom Sheet > Show Bottom Sheet** (or **Alert Dialog > Custom Dialog**).



3. Select your newly created component from the dropdown.
4. **Pass the Data:** You will see your Component Parameter listed. Click it, and link it to the specific row/document context of the list item the user just tapped.



3. Mastering Local Storage: App State vs. Page State vs. Component State

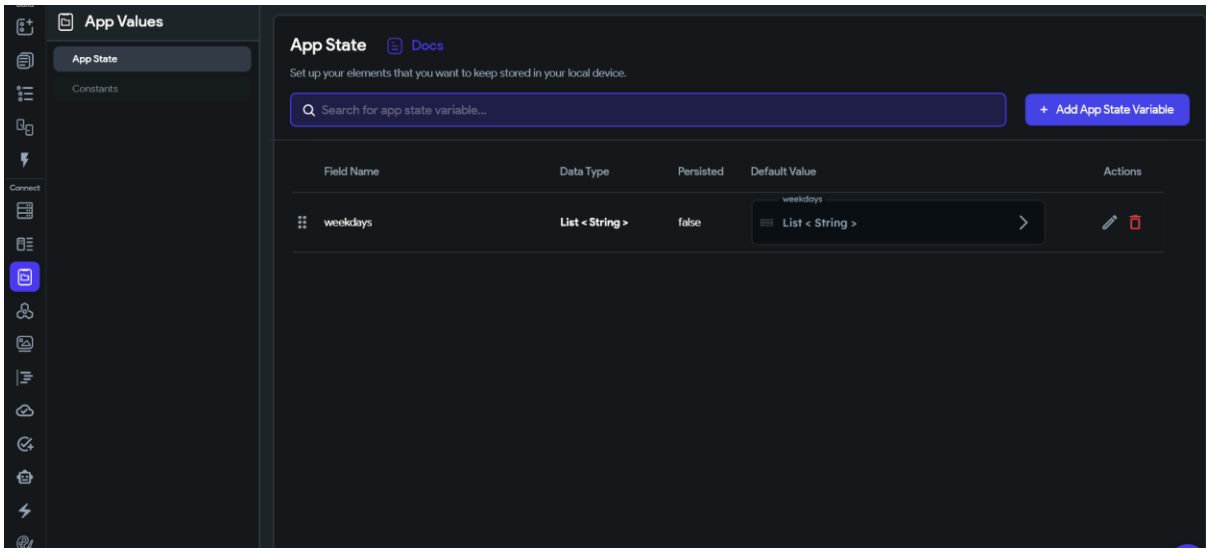
State management dictates how your app remembers information. FlutterFlow provides three distinct scopes of local storage, each tailored to specific lifecycles.

Storage Scope	Lifespan	Accessibility	Ideal Use Case

App State	Global (Persists across pages until app closes)	Accessible by **any** page or component	Shopping carts, user preferences, auth tokens
Page State	Temporary (Destroyed when page is exited)	Accessible **only** within that specific page	Search queries, active filters, form draft steps
Component State	Temporary (Destroyed when popup closes)	Accessible **only** inside that exact component	Counter buttons inside a list card, temporary dialog selections

Deep Dive: App State (Global & Permanent Storage)

App State variables hold data that needs to be accessed globally.



Configuration Properties:

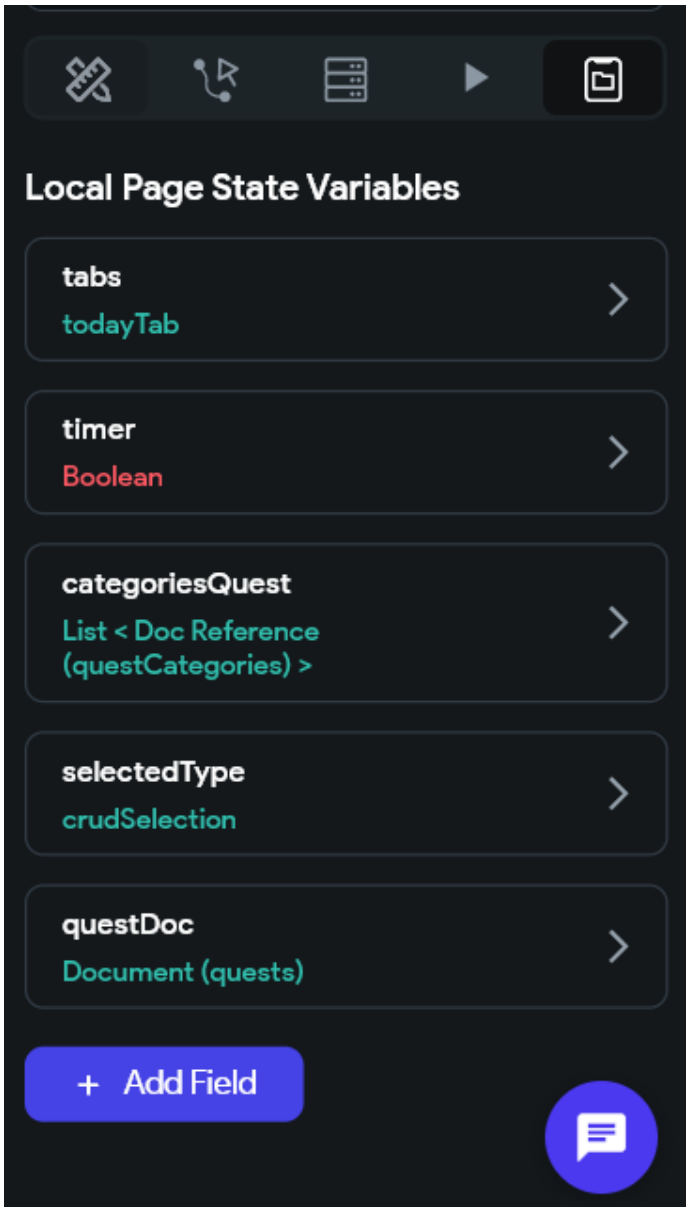
- **Is List:** Toggle this on if your variable needs to hold multiple items (e.g., a list of product IDs in a shopping cart).
- **Persisted:** Turning this on saves the data **directly to the device's local disk**. If the user closes the app entirely and reopens it tomorrow, a persisted App State variable will still remember its data.

Rebuild Options on Update:
When you use the **Update App State** action, you must choose an **Update Type**:

- **Rebuild All Pages:** Refreshes the UI across your entire app (use sparingly, as it can cause performance hitches).
- **Rebuild Current Page:** Refreshes only the active screen the user is looking at.
- **No Rebuild:** Updates the data behind the scenes silently without causing a visual flicker.

Deep Dive: Page State & Component State (Temporary Storage)

Page State handles transient data that doesn't matter to the rest of your app.



Setup Configuration:

- **Nullable:** If set to `True`, the variable can start entirely empty (`null`). If `False`, you must supply a rigid **Initial Field Value** (e.g., setting a `searchString` variable to an empty text string `""` so it doesn't cause errors on page load).

Rebuild on Update:

Just like App State, when using the **Update Page State** or **Update Component State** actions, always choose **Rebuild Current Page** if the UI needs to instantly visually transform based on the new data (such as showing/hiding elements dynamically).

Best Practices Summary Checklist

1. **Use Page State for Inputs:** Never use App State to store active text inputs from a form. Use **Page State** to avoid slowing down your global application performance.
2. **Enable Persistence Judiciously:** Only toggle **Persisted** on for App State variables that *actually* need to survive an app restart (like dark mode toggles or cached user session details).
3. **Clean Up with Views and Simple Search:** When dealing with searches, map your local simple search to a clean, pre-flattened list database view to save processing cycles on the user's mobile device.